If you have multiple group blocks with the same group name, the group adds the contents of all of the blocks. In the above example, the :development group contains Standard, debug, and RSpec, while the :test group contains only RSpec.

To use groups, you modify the Bundle setup, rather than require bundle/setup, you just require bundle and then explicitly call Bundler.setup with one or more group names.

```
require 'bundler'
Bundler.setup(:development, :test)
```

Gems that are not a part of any group will always be installed, calling setup with arguments additionally adds the gems that are in the specified groups.

If you are using bundle exec to load a program, the default will be to install all gems in all groups, but you can modify this with the environment variables BUNDLE_WITH or BUNDLE_WITHOUT—BUNDLE_WITH creates a list of groups to include and ignores other groups, BUNDLE_WITHOUT creates a list of groups to exclude and includes all other groups.

```
$ BUNDLE_WITH=test bundle exec rspec
```

If one of the environment variables is set, then groups will only be loaded if they conform to the logic of the environment variables.

# Writing and Packaging Your Own Code Into Gems

RubyGems is not just for downloading gems; you can also write and distribute your own gems. Even if you don't plan to distribute a gem, the default packaging for RubyGems can help you plan the basic structure of your Ruby code. There used to be multiple sources for how to structure a Ruby gem, but Bundler also provides a default template that has become the basic standard.

As your programs grow (and they all seem to grow over time), you'll find that you'll need to start organizing your code—simply putting everything into a single huge file becomes unworkable (and makes it hard to reuse chunks of code in other projects). So, we need to find a way to split our project into multiple files and then to knit those files together as our program runs.

There are two major aspects to this organization. The first is internal to your code: how do you prevent different things with the same name from clashing? The second area is related: how do you conveniently organize the source files in your project?

## Single File Projects

Small, self-contained scripts can be in a single file. However, if you do this, you won't easily be able to write automated tests for your program, because the test code won't be able to load the file containing your source without the program itself running. So, if you want to write a small program that also has automated tests, split that program into a trivial driver that provides the external interface (the command-line part of the code) and one or more files containing the rest. Your tests can then exercise these separate files without actually running the main body of your program.

Let's try this for real. Here's a simple program that finds anagrams in a dictionary. Feed it one or more words, and it gives you the anagrams of each. Here's an example:

```
$ ruby anagram.rb teaching ruby
```